Análise Empírica de Desempenho e Complexidade de Algoritmos de Ordenação com Java

Bruno Benitez Forgiarini[1], **Maria Eduarda Maia Almeida**[1]

[1]Universidade Federal do Pampa (Unipampa)

brunoforgiarini.aluno@unipampa.edu.br

mariaema2.aluno@unipampa.edu.br

Abstract. *This report presents a comprehensive empirical study on the computational efficiency of ten sorting algorithms (BubbleSort, SelectionSort, InsertionSort, ShellSort, MergeSort, QuickSort, HeapSort, CountingSort, RadixSort, and BucketSort) implemented in Java. The experiment evaluated the impact of asymptotic complexities $O(n^2)$, $O(n \log n)$, and $O(n)$, initial data distributions (Random, Sorted, Reversed, and Repeated), and the Object-Oriented overhead across different data types (`Integer`, `String`, and a custom `Pessoa` class). The results provide empirical evidence of the differences in scaling behavior between complexity classes and illustrate how data types and implementation choices can influence execution time in the JVM.*

Resumo. *Este relatório apresenta um estudo empírico abrangente sobre a eficiência computacional de dez algoritmos de ordenação (BubbleSort, SelectionSort, InsertionSort, ShellSort, MergeSort, QuickSort, HeapSort, CountingSort, RadixSort e BucketSort) implementados na linguagem Java. O experimento avaliou o impacto das classes de complexidade assintótica $O(n^2)$, $O(n \log n)$ e $O(n)$, das distribuições iniciais dos dados (Aleatório, Ordenado, Invertido e Repetidos) e do impacto do tipo de dado e da representação utilizada no desempenho através de tipos distintos (`Integer`, `String` e a classe genérica `Pessoa`). Os resultados evidenciam experimentalmente a divergência de escala entre as classes de complexidade e ilustram como os tipos de dados e as escolhas de implementação podem influenciar o desempenho na JVM.*

1. Introdução

Este relatório técnico apresenta o desenvolvimento, a verificação e a análise experimental de desempenho de dez algoritmos de ordenação, desenvolvidos como parte do Trabalho Prático 1 (TP1) da disciplina de Algoritmos e Classificação de Dados do curso de Engenharia de Computação da UNIPAMPA. O escopo principal deste estudo é analisar os métodos de ordenação de dados para o propósito de avaliação, com respeito ao tempo de execução e ao comportamento de desempenho na prática, a partir da perspectiva do desenvolvedor.

Para conduzir essa investigação, foram implementados em Java sete algoritmos baseados em comparação (Bubble Sort, Insertion Sort, Selection Sort, Shell Sort, Heap Sort, Merge Sort e Quick Sort) e três algoritmos de tempo linear (Counting Sort, Radix Sort e Bucket Sort). A avaliação experimental submeteu esses algoritmos a diferentes cenários de estresse, variando o tamanho das coleções (de 100 até 1.000.000 de elementos), a

disposição inicial dos dados (ordem aleatória, crescente, invertida e com várias ocorrências repetidas) e os tipos de dados processados (tipos primitivos como `int[]` e objetos como `Integer`, `String` e uma classe customizada `Pessoa`).

O objetivo central desta análise vai além da simples identificação dos menores tempos absolutos de execução. Busca-se interpretar os dados coletados (totalizando 4.440 execuções válidas) para identificar as características das entradas que mais influenciam a eficiência de cada método e relacionar diretamente os resultados observados na prática com a análise teórica de complexidade assintótica estudada ao longo do componente curricular.

2. Fundamentação Teórica e Algoritmos Analisados

Modelo RAM e Notação Assintótica

A análise da complexidade dos algoritmos baseia-se no modelo *Random Access Machine* (RAM), que assume que instruções básicas (como operações aritméticas, atribuições e comparações) são executadas em tempo constante. A notação assintótica Big-O é utilizada para expressar o limite superior do tempo de execução em função do tamanho da entrada n. Para algoritmos de ordenação estritamente baseados em comparações, a teoria estabelece um limite inferior de $\Omega(n \log n)$ para o pior caso. Um resumo das complexidades teóricas dos algoritmos avaliados está expresso na Tabela 1.

Tabela 1. Complexidade Teórica dos Algoritmos de Ordenação Estudados

Algoritmo	Melhor Caso	Caso Médio	Pior Caso	Espaço Auxiliar	Estável?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sim
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	Não
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sim
Shell Sort	$O(n \log n)$	$O(n^{1.5})$	$O(n^2)$	$O(1)$	Não
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Sim
Quick Sort (3 Vias)	$O(n)$ / $O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	Não
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	Não
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(n + k)$	Sim
Radix Sort	$O(d(n + k))$	$O(d(n + k))$	$O(d(n + k))$	$O(n + k)$	Sim
Bucket Sort	$O(n + k)$	$O(n + k)$	$O(n \log n + k)$ *	$O(n + k)$	Sim

Nota (*): Na implementação avaliada, o pior caso do *Bucket Sort* é limitado a $O(n \log n + k)$ devido ao uso do *Merge Sort* na ordenação interna dos baldes (em implementações clássicas com *Insertion Sort*, o pior caso é $O(n^2)$).

Algoritmos Comparativos

Os algoritmos com complexidade de pior caso $O(n^2)$ implementam lógicas simples e apresentam bom desempenho apenas para coleções pequenas ou já quase ordenadas:

- **Bubble Sort:** Percorre repetidamente o vetor, comparando elementos adjacentes e trocando suas posições quando estão fora de ordem. Em cada passagem, os maiores elementos da região percorrida tendem a deslocar-se para o final do vetor. O processo é repetido até que os elementos estejam ordenados. Sua implementação possui complexidade de tempo $O(n^2)$ no pior caso e utiliza espaço auxiliar $O(1)$.
- **Selection Sort:** Divide conceitualmente o vetor em uma região ordenada e outra não ordenada. A cada iteração, procura o menor elemento da região não ordenada

e o troca com o primeiro elemento dessa região. Ao final de cada etapa, mais uma posição passa a integrar a região ordenada. O algoritmo realiza $\Theta(n^2)$ comparações mesmo quando a entrada já está ordenada, utiliza espaço auxiliar $O(1)$ e realiza poucas trocas em comparação com outros métodos simples.
- **Insertion Sort:** Remove sequencialmente cada item da parte não ordenada e o insere na posição correta dentro da sublista previamente ordenada.

Algoritmos Comparativos Avançados

Esses algoritmos utilizam estratégias que reduzem o custo da ordenação ou organizam os elementos por intervalos e estruturas auxiliares.

- **Shell Sort:** Extensão do *Insertion Sort* que utiliza intervalos decrescentes. Nesta implementação, adota a sequência de Knuth ($h = 3h + 1$). Sua complexidade depende da sequência utilizada.
- **Merge Sort:** Divide recursivamente o vetor e intercala as partes ordenadas. Apresenta tempo $O(n \log n)$ e espaço auxiliar $O(n)$.
- **Quick Sort com Particionamento de 3 Vias:** Utiliza o elemento central como pivô e separa valores menores, iguais e maiores que ele. Ordena recursivamente o menor intervalo e processa o maior por laço, reduzindo a profundidade da pilha. Seu pior caso continua sendo $O(n^2)$.
- **Heap Sort:** Utiliza um *max-heap* para selecionar repetidamente o maior elemento. Garante tempo $O(n \log n)$ e espaço auxiliar $O(1)$ na implementação *in-place*.

Algoritmos Não Comparativos

Esses algoritmos exploram características dos valores de entrada, como amplitude numérica e quantidade de dígitos, evitando comparações diretas entre chaves:

- **Counting Sort:** Conta as ocorrências dos valores em um vetor cujo tamanho corresponde à amplitude da entrada ($k = \text{maior} - \text{menor} + 1$), determinando diretamente a posição final de cada elemento. Apresenta complexidade de tempo e espaço auxiliar $O(n + k)$, onde n é o número de elementos e k é a amplitude numérica considerada, suportando números negativos por meio de deslocamento (*offset*).
- **Radix Sort:** Processa os números inteiros em base decimal fixa, ordenando os elementos dígito a dígito por meio de uma versão estável do *Counting Sort* com vetor de contagem fixo de 10 posições e vetor auxiliar de tamanho n. Sua complexidade temporal é $O(d(n + 10))$, equivalente a $O(dn)$, com consumo de memória auxiliar $O(n + 10)$ por passagem, em que d representa a quantidade de posições decimais. Diferentemente do *Counting Sort*, não aloca memória proporcional à amplitude total dos dados, exigindo inteiros não negativos e vetor não vazio.
- **Bucket Sort:** Distribui os elementos em n baldes (`ArrayList<Integer>`), calculando o índice de destino proporcionalmente ao intervalo dos valores (intervalo $= \lfloor(\text{maior} - \text{menor})/n\rfloor + 1$). Em seguida, cada balde contendo dois ou mais elementos (tamanho ≥ 2) é ordenado individualmente por um método auxiliar próprio (`ordenarBalde`) que implementa o algoritmo *Merge Sort* recursivo com lista auxiliar. Por fim, os baldes são concatenados sequencialmente no arranjo original.

Para um balde contendo b elementos, a ordenação interna apresenta complexidade temporal $O(b \log b)$ e demanda espaço auxiliar $O(b)$. Considerando a distribuição inicial dos n elementos em k baldes, o custo total depende da quantidade de elementos concentrados em cada estrutura. Em distribuições aproximadamente uniformes, nas quais os baldes apresentam ocupação reduzida, o desempenho aproxima-se de $O(n + k)$. Por outro lado, quando uma parcela significativa dos elementos concentra-se em poucos baldes, o custo da ordenação interna aumenta, podendo atingir $O(n \log n + k)$ no pior caso. Essa característica diferencia a implementação utilizada de versões do *Bucket Sort* que empregam algoritmos de ordenação interna com complexidade quadrática.

3. Metodologia Experimental

A avaliação experimental foi planejada e executada de forma automatizada, buscando favorecer a reprodutibilidade dos resultados e reduzir possíveis interferências externas durante as medições.

3.1. Pipeline de Desenvolvimento e Verificação

O fluxo de desenvolvimento foi dividido nas seguintes etapas sequenciais:

1. **Implementação dos Algoritmos:** Construção das classes dos dez algoritmos de ordenação na linguagem Java, utilizando a interface `Comparable<T>` para genéricos e arranjos `int[]` para algoritmos lineares.
2. **Testes de Corretude (`TestesCorretude`):** Desenvolvimento de testes automatizados para validar a exatidão dos dez algoritmos em vetores reduzidos (entre 10 e 15 elementos, contemplando tamanhos pares e ímpares) sob quatro disposições de entrada (aleatório, já ordenado, invertido e com repetidos), confrontando a saída obtida com o gabarito oficial do método `Arrays.sort()` da biblioteca padrão do Java.
3. **Gerador de Entradas (`GeradorDeEntradas`):** Módulo responsável por instanciar os arranjos. Para garantir a reprodutibilidade, utilizou-se `java.util.Random` com semente fixa (`seed = 42`), estruturando os cenários da seguinte forma:
    - **Configurações de Entrada:** O cenário *Aleatório* sorteia chaves no intervalo $[0, 10n)$; o *Ordenado* preenche o vetor de forma crescente ($i = 0 \ldots n-1$); o *Invertido* dispõe os valores em ordem decrescente ($n-1-i$); e o *Repetidos* sorteia números estritamente no intervalo $[0, 10)$, garantindo alta taxa de duplicatas.
    - **Adaptação por Tipo de Dado:** Nos vetores de `Integer` e `int[]`, os valores gerados são atribuídos diretamente. Para `String`, os números são formatados com máscara fixa de 8 dígitos (`"STR_%08d"`), assegurando que a ordenação lexicográfica reflita exatamente a grandeza numérica. Para a classe `Pessoa`, a chave define o atributo comparável (`id`), enquanto a "carga útil" do objeto (`nome`, `idade` e `salario`) é preenchida deterministicamente a partir do índice.
4. **Motor de Benchmarking (`BenchmarkRunner`):** Executável principal encarregado de orquestrar os aquecimentos da JVM, clonar os vetores a cada iteração, cronometrar a ordenação via `System.nanoTime()` e exportar os dados brutos.

3.2. Parâmetros do Experimento e Tipos de Dados

O experimento avaliou o desempenho combinando os seguintes fatores:

- **Tamanhos de Entrada (N):** Vetores variando de 100 até 1.000.000 elementos. Devido ao tempo proibitivo (onde algoritmos quadráticos superaram 35 segundos para $N = 100.000$), *BubbleSort*, *SelectionSort* e *InsertionSort* foram **excluídos do cenário** $N = 1.000.000$.
- **Cenários de Distribuição Inicial:** Aleatório, Ordenado, Invertido e Repetidos (alta densidade de chaves duplicadas). Essas configurações permitem avaliar a influência da disposição inicial dos dados sobre os algoritmos, embora não representem necessariamente o melhor ou o pior caso teórico de todos eles.
- **Tipos de Dados:** O experimento cobriu arranjos primitivos (`int[]`), encapsuladores genéricos puros (`Integer`), cadeias léxicas (`String` no formato `"STR_%08d"` com custo interno $O(L)$) e objetos estruturados na *Heap* (`Pessoa`).

3.3. Sanitização, Tratamento de Dados e Análise Estatística

Antes do início das medições cronometradas, a suíte executou uma bateria global de pré-aquecimento da JVM (*warm-up*) composta por 3 rodadas completas sobre todos os dez algoritmos utilizando arranjos aleatórios de tamanho reduzido ($N = 128$), descartadas da contagem, seguidas pelas 10 repetições oficiais para cada cenário experimental.

Os dados brutos coletados pelo `BenchmarkRunner` foram exportados em milissegundos (ms). O arquivo de resultados passou por uma etapa automatizada de tratamento através de um script em Python, utilizando as bibliotecas *Pandas* e *Matplotlib* para a geração do resumo estatístico e dos gráficos, respectivamente. A partir das 10 medições de cada configuração, calculou-se o resumo estatístico completo contendo: Média (μ), Desvio Padrão (σ), Mediana (Q_2), valores de Mínimo e Máximo (Amplitude) e Quartis (Q_1, Q_3). Na Tabela 2, apresentamos de forma consolidada as métricas centrais (μ, σ e Q_2) para os maiores volumes de dados.

3.4. Ambiente de Hardware e Software

O benchmark foi executado em ambiente dedicado (AMD Ryzen™ 5 5600 3.5 GHz, 16 GiB RAM DDR4, SSD 1.7 TB) rodando Ubuntu 24.04.5 LTS (Kernel Linux 7.0.0). Os testes operaram na OpenJDK 17 sem atividades secundárias relevantes em segundo plano durante a execução.

4. Resultados e Discussão

A avaliação gerou um vasto conjunto de métricas empíricas. Os resultados foram agrupados considerando escalas de tamanho e tipos de dados, utilizando as médias da Tabela 2 como referência exata para as análises gráficas.

Tabela 2. Resumo Estatístico Consolidado: Cenário Aleatório ($N = 100.000$ e $N = 1.000.000$)

Algoritmo	Tipo	Tempo Médio (ms) (μ)	Desvio Padrão (σ)	Mediana (Q2)
Tamanho $N = 100.000$				
BubbleSort	`Integer`	36289,66	57,92	36271,01
HeapSort	`Integer`	24,41	0,07	24,42
InsertionSort	`Integer`	13573,81	13,25	13571,89
MergeSort	`Integer`	17,36	0,13	17,33
QuickSort	`Integer`	18,67	0,66	18,38
SelectionSort	`Integer`	23870,97	23,84	23867,88
ShellSort	`Integer`	29,90	0,22	29,81
BubbleSort	`Pessoa`	38745,55	53,86	38730,26
HeapSort	`Pessoa`	26,38	0,07	26,37
InsertionSort	`Pessoa`	14724,36	24,46	14718,51
MergeSort	`Pessoa`	18,71	0,03	18,72
QuickSort	`Pessoa`	20,97	0,60	20,79
SelectionSort	`Pessoa`	26198,14	132,80	26221,40
ShellSort	`Pessoa`	31,57	0,10	31,56
BubbleSort	`String`	89069,35	128,19	89108,85
HeapSort	`String`	49,95	0,22	49,90
InsertionSort	`String`	27980,66	12,51	27981,57
MergeSort	`String`	30,63	0,04	30,64
QuickSort	`String`	36,15	0,89	35,86
SelectionSort	`String`	51770,04	1351,77	52178,71
ShellSort	`String`	65,09	1,87	64,43
BucketSort	`int[]`	18,46	6,31	16,99
CountingSort	`int[]`	3,45	0,08	3,45
RadixSort	`int[]`	2,45	0,03	2,47
Tamanho $N = 1.000.000$				
HeapSort	`Integer`	396,72	1,38	396,59
MergeSort	`Integer`	281,11	27,56	270,35
QuickSort	`Integer`	320,36	3,34	319,40
ShellSort	`Integer`	557,80	1,24	557,83
HeapSort	`Pessoa`	632,57	8,56	632,16
MergeSort	`Pessoa`	409,34	4,95	407,77
QuickSort	`Pessoa`	498,96	39,68	501,33
ShellSort	`Pessoa`	1052,10	6,24	1051,37
HeapSort	`String`	945,29	4,26	944,53
MergeSort	`String`	501,13	2,40	500,52
QuickSort	`String`	576,17	4,34	575,08
ShellSort	`String`	1694,27	12,18	1693,26
BucketSort	`int[]`	209,85	28,78	216,14
CountingSort	`int[]`	49,77	0,74	49,68
RadixSort	`int[]`	31,76	5,59	28,93

4.1. Comparação Geral de Classes de Complexidade

Para estabelecer uma linha de base, avaliou-se o caso médio (vetores com disposição aleatória) dividindo a análise entre os algoritmos comparativos e os de tempo linear.

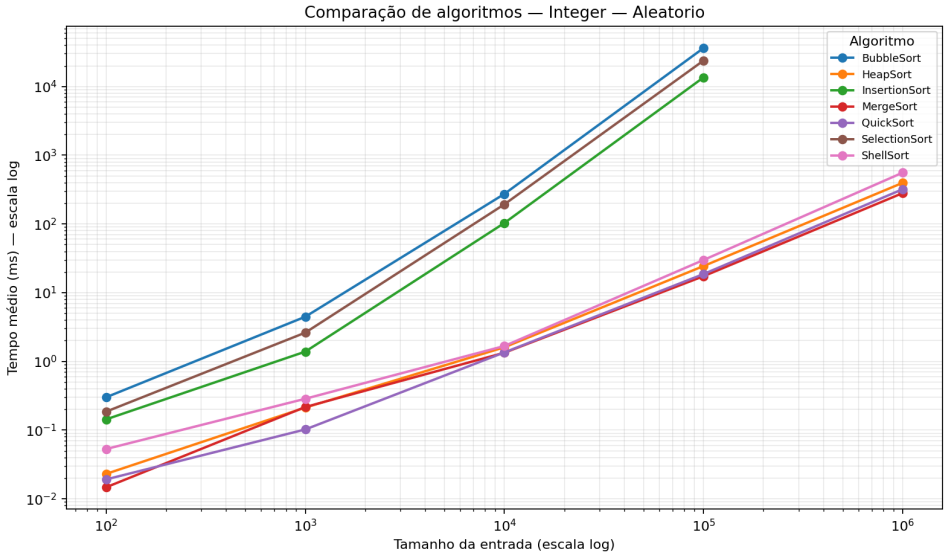


Figura 1. Comparação geral em arranjos aleatórios para algoritmos baseados em comparação (`Integer`).

Conforme suportado pela Tabela 2, a Figura 1 apresenta uma divisão clara entre as complexidades. Para $N = 100.000$, algoritmos $O(n^2)$ como *Bubble Sort* sofrem uma escalada abrupta de tempo (chegando a 36.289 ms). Em contrapartida, os $O(n \log n)$ mantêm uma curva de crescimento suave (com o *Merge Sort* cravando $17{,}36$ ms).

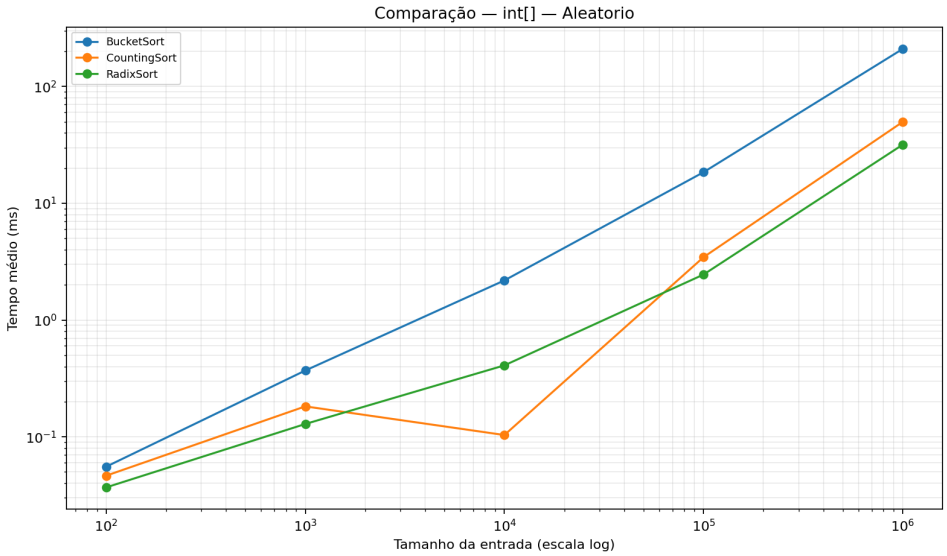


Figura 2. Comparação geral em arranjos aleatórios para algoritmos de tempo linear (`int[]`).

A Figura 2 ilustra a execução sobre `int[]`, onde operam os algoritmos não-comparativos. Evidencia-se a eficácia dessas abordagens espaciais, que alcançam tempos substancialmente reduzidos (o *RadixSort* registrou $2{,}45$ ms para $N = 100.000$ e apenas $31{,}76$ ms para $N = 1.000.000$, conforme a Tabela 2). Nota-se uma inflexão atípica na

curva do *CountingSort* entre 10^3 e 10^4 elementos, fenômeno decorrente da compilação em camadas da JVM que será detalhado na análise de perfil individual (Seção 4.5).

4.2. Sensibilidade ao Estado Inicial e Casos Limites

O grau de ordenação afeta drasticamente o desempenho empírico.

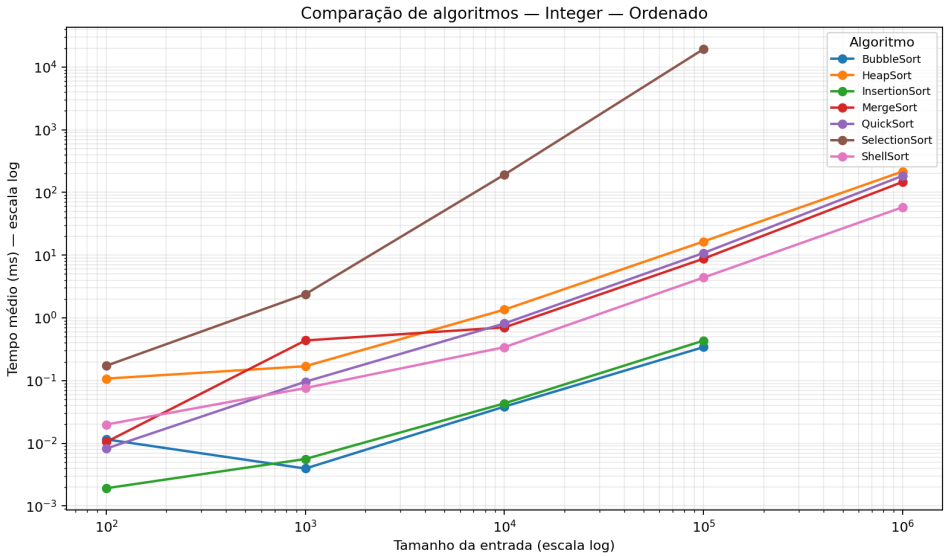


Figura 3. Impacto do estado inicial: Melhor Caso (vetor previamente Ordenado).

Na Figura 3, destaca-se a forte adaptabilidade do *InsertionSort* e do *BubbleSort* em vetores já ordenados. Este desempenho excepcional do *BubbleSort* deve-se à sua implementação otimizada: a utilização de uma *flag* booleana de controle (`trocou = false`). Como não ocorrem trocas na primeira passagem, o laço é interrompido precocemente, garantindo a ambos os algoritmos um custo linear $O(n)$ e máxima eficiência. Em contraste, o *SelectionSort* não é adaptativo: ignora a ordenação prévia, mantém o volume máximo de comparações ($O(n^2)$) e apresenta a maior latência do gráfico.

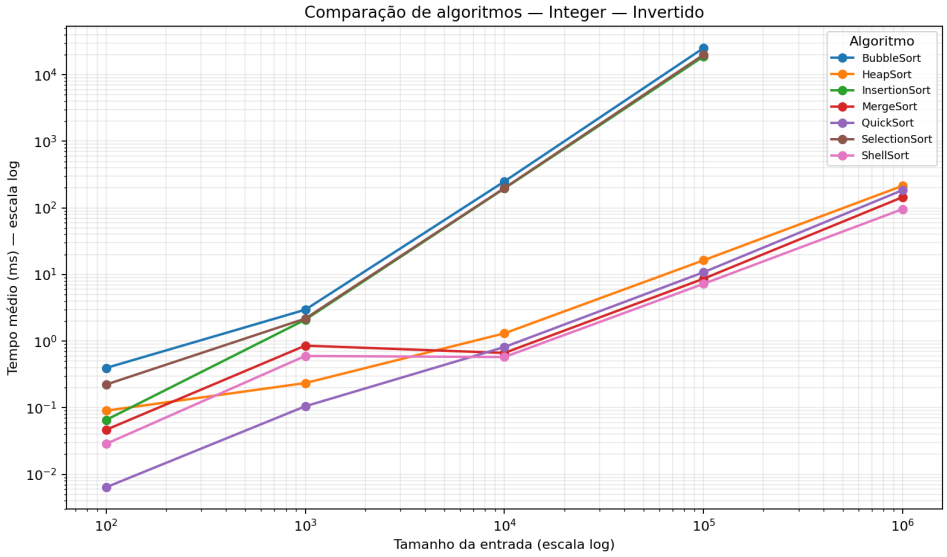


Figura 4. Impacto do estado inicial: vetor em ordem inversa (`Integer`).

A Figura 4 apresenta os resultados para entradas em ordem inversa, configuração que representa um cenário desfavorável (pior caso) para algoritmos adaptativos como *BubbleSort* e *InsertionSort*. Nesses métodos, a otimização de parada precoce não é acionada e o número de movimentações cresce significativamente em relação ao cenário ordenado, degradando o tempo para $O(n^2)$. Para os demais algoritmos (*MergeSort*, *HeapSort*, *QuickSort* e *ShellSort*), o efeito depende de suas estratégias de divisão, seleção de pivô e estruturas auxiliares, mantendo curvas estáveis próximas a $O(n \log n)$ nas implementações avaliadas.

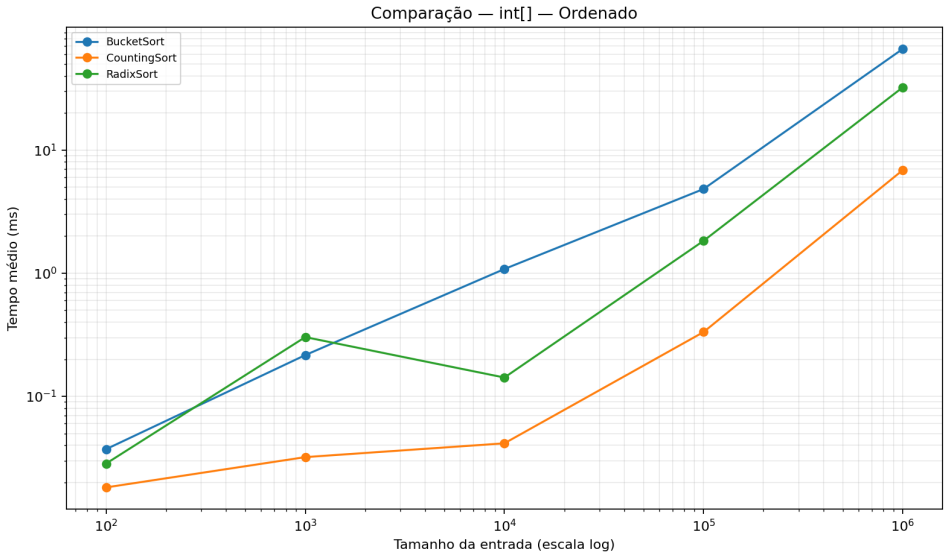


Figura 5. Impacto do estado inicial em algoritmos não comparativos: vetor Ordenado (`int[]`).

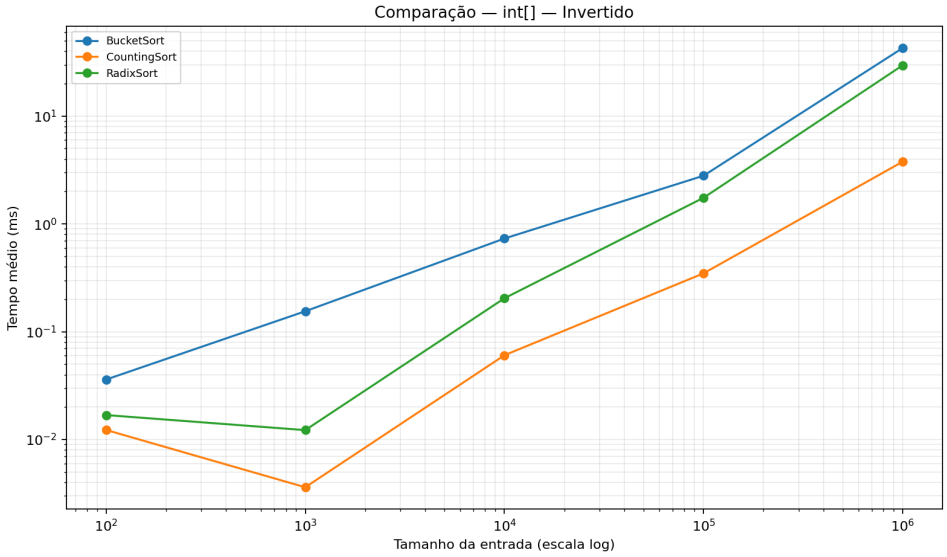


Figura 6. Impacto do estado inicial em algoritmos não comparativos: vetor Invertido (`int[]`).

As Figuras 5 e 6 ilustram o desempenho dos algoritmos não comparativos (*BucketSort*, *CountingSort* e *RadixSort*) operando sobre tipos primitivos (`int[]`) nos cenários de entrada Ordenado e Invertido.

Nos cenários ordenado e invertido, *CountingSort* e *RadixSort* apresentaram comportamentos próximos, uma vez que suas etapas de processamento dependem principalmente da amplitude do domínio numérico e da quantidade de dígitos dos elementos, respectivamente, e não diretamente da ordem inicial dos valores. O *BucketSort*, por sua vez, apresentou variações associadas à distribuição dos elementos entre os baldes e à necessidade de ordenação interna: enquanto entradas ordenadas e invertidas distribuem exatamente um elemento por balde, dispensando a execução do *MergeSort* auxiliar, entradas aleatórias ou concentradas podem exigir processamento adicional devido à maior ocupação de determinados baldes.

Assim, embora os três algoritmos utilizem estratégias baseadas na distribuição ou representação das chaves, suas implementações respondem de maneiras distintas às características da entrada. Os resultados indicam que não existe um único algoritmo com menor latência em todas as configurações avaliadas. Embora o *CountingSort* tenha apresentado desempenho competitivo, o *RadixSort* obteve menores tempos médios no cenário Aleatório para $N = 100\,000$ ($2{,}45$ ms contra $3{,}45$ ms) e $N = 1\,000\,000$ ($31{,}76$ ms contra $49{,}77$ ms), conforme a Tabela 2. Dessa forma, a eficiência observada depende não apenas da complexidade assintótica, mas também das características das entradas e dos custos específicos de implementação.

4.3. Impacto do Tipo de Dado e Cenários de Entrada

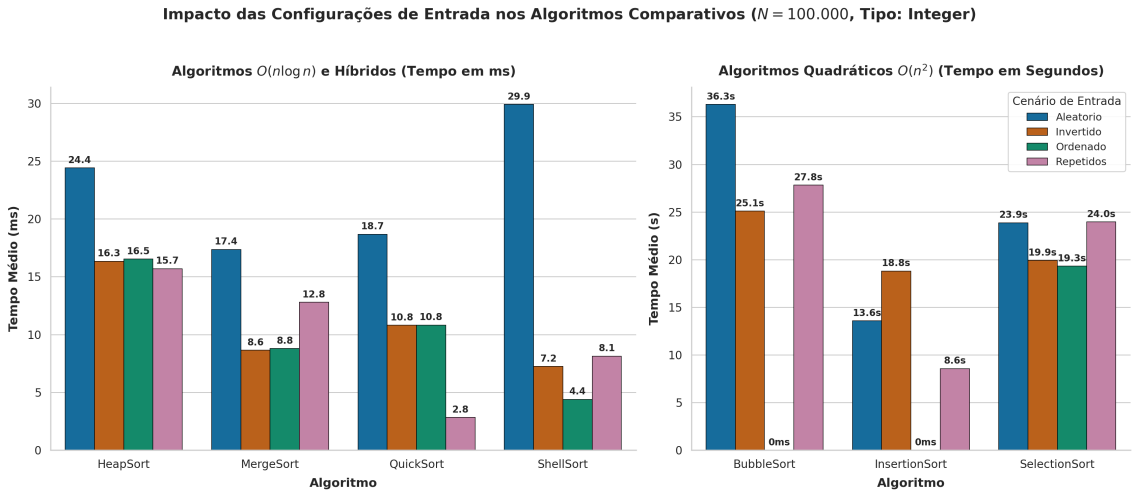


Figura 7. Desempenho nos casos Aleatório, Invertido, Ordenado e Repetido com ordenação do tipo `Integer`.

A Figura 7 consolida o comportamento dos algoritmos sob quatro configurações iniciais distintas para o tipo `Integer`. Nota-se que a disposição prévia dos dados dita drasticamente o desempenho das estruturas elementares: enquanto o cenário **Ordenado** representa o melhor caso para algoritmos adaptativos (como *InsertionSort* e *BubbleSort*), reduzindo o custo computacional para $O(n)$, o cenário **Invertido** impõe a latência máxima ao forçar o pior caso de movimentações na memória ($O(n^2)$).

Por outro lado, o cenário **Aleatório** reflete o caso médio de execução esperado. Já as entradas com elementos **Repetidos** demonstram latências inferiores ao caso aleatório, favorecendo especialmente algoritmos mais sofisticados (como o *QuickSort*), que tiram proveito de otimizações de particionamento e da drástica redução no número de chaves únicas a serem movimentadas.

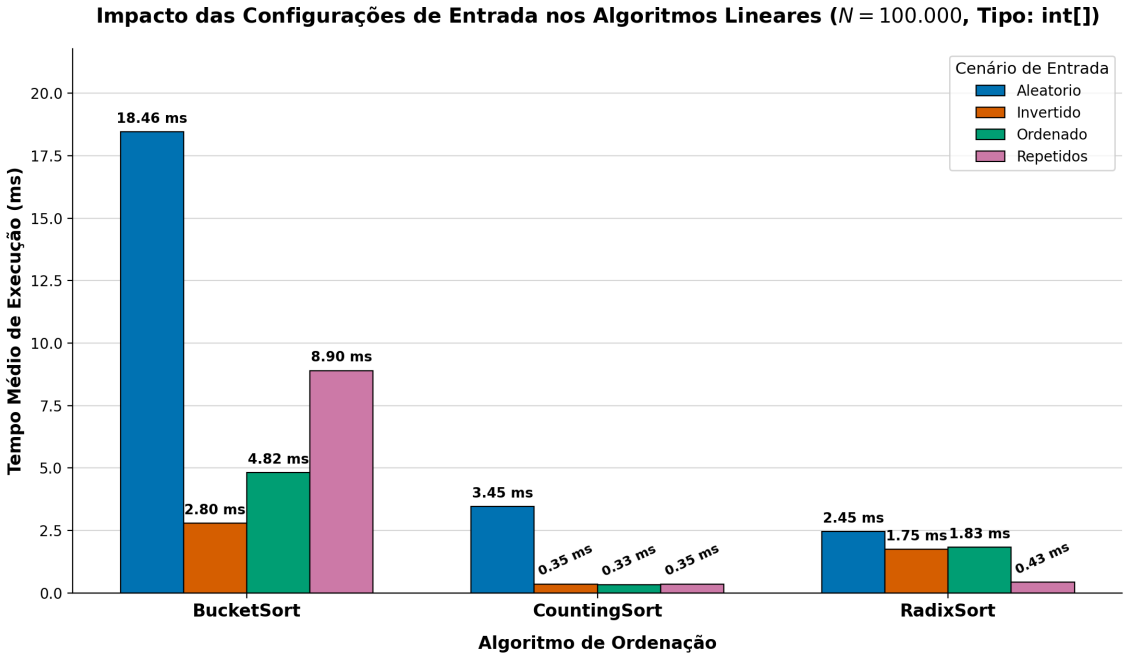


Figura 8. Impacto dos cenários de entrada nos algoritmos de complexidade linear ($N = 100.000$, tipo primitivo `int[]`).

A Figura 8 ilustra o tempo médio de execução dos três algoritmos não comparativos (*CountingSort*, *RadixSort* e *BucketSort*) operando sobre `int[]` para $N = 100.000$ sob as quatro configurações iniciais. Observa-se que o cenário **Aleatório** impõe a maior latência para todos os três métodos ($3{,}45$ ms no *CountingSort*, $2{,}45$ ms no *RadixSort* e $18{,}46$ ms no *BucketSort*). Esse comportamento reflete diretamente as características das entradas e das implementações:

- No ***CountingSort*** e no ***RadixSort***, o domínio mais amplo $[0, 10n)$ do cenário **Aleatório** exige um vetor de contagem dez vezes maior ($k \approx 10^6$) e seis passagens de dígitos ($d = 6$), ao passo que no cenário **Repetidos** (domínio $[0, 10)$) a amplitude cai para $k \leq 10$ e exige apenas uma passagem ($d = 1$, com $0{,}43$ ms no *RadixSort*).
- No ***BucketSort***, os menores tempos ocorreram nos cenários **Invertido** ($2{,}80$ ms) e **Ordenado** ($4{,}82$ ms), nos quais a distribuição uniforme aloca exatamente 1 elemento por balde, acionando a condição de parada imediata (`balde.size() < 2`) sem instanciar listas auxiliares. Em contrapartida, no cenário **Repetidos** ($8{,}90$ ms), a baixa amplitude concentra os 100.000 elementos em apenas 10 baldes (≈ 10.000 chaves por balde), acionando a ordenação interna via *Merge Sort* ($\Theta(b \log b)$) e cópias em listas auxiliares. Já no caso **Aleatório** ($18{,}46$ ms), somam-se as colisões estatísticas em múltiplos baldes, a alocação dinâmica de listas auxiliares na *Heap* (`ArrayList<Integer>`) e o custo de *autoboxing* de tipos primitivos.

É importante observar que os cenários não diferem exclusivamente pela ordem inicial dos elementos. As configurações Aleatório e Repetidos também utilizam domínios numéricos distintos, o que altera a amplitude dos valores e a quantidade de chaves distintas. Assim, as diferenças de desempenho observadas refletem conjuntamente a disposição dos dados e as características de seus domínios.

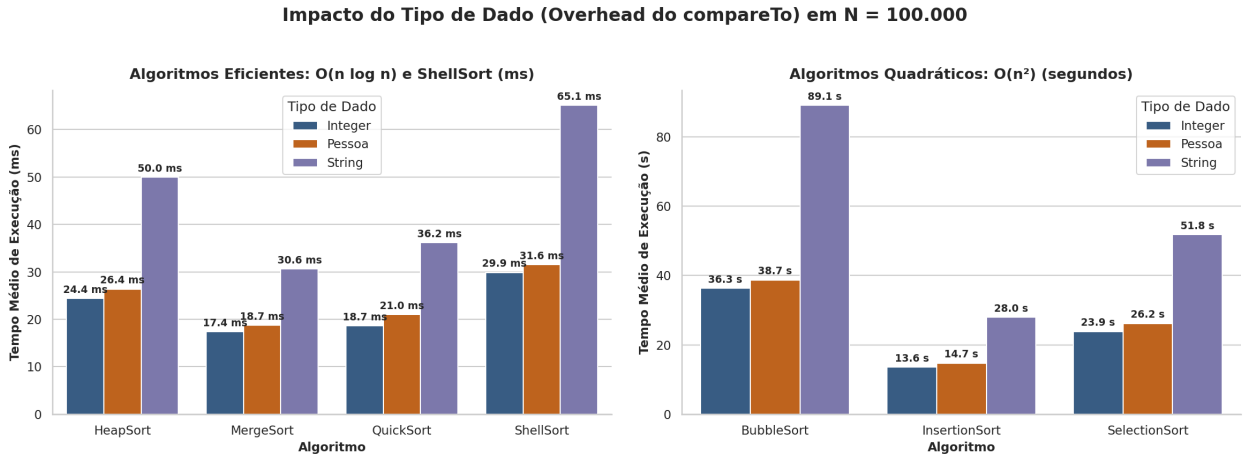


Figura 9. Análise do impacto e sobrecarga de abstração entre diferentes tipos de dados (`Integer`, `Pessoa` e cadeias de caracteres `String`).

Ao comparar a Tabela 2 e a Figura 9, observa-se que o *QuickSort* apresentou tempos próximos para `Integer` e `Pessoa` (18,67 ms e 20,97 ms, respectivamente, uma diferença de $\approx 12{,}3\%$). Essa proximidade pode estar relacionada às operações realizadas durante a comparação e à representação dos elementos: na implementação de `Pessoa`, o método `compareTo()` utiliza exclusivamente o identificador numérico (`id`) como chave em tempo $O(1)$, enquanto as operações de movimentação reorganizam referências aos objetos na *Heap*, sem copiar seus demais atributos (`nome`, `idade` e `salario`). Já a ordenação de objetos `String` apresentou tempos superiores, especialmente nos algoritmos que realizam grande quantidade de comparações, aspecto discutido na Seção 4.6.

4.4. Análise de Dispersão e Variabilidade Estatística

A partir dos dados consolidados na Tabela 2 e nas estatísticas globais coletadas, a análise das métricas de centralidade e dispersão (Média μ, Desvio Padrão σ, Mediana Q_2 e Amplitude) revela comportamentos fundamentais sobre a estabilidade dos algoritmos e a dinâmica de execução na JVM:

1. **Baixa Variabilidade dos Algoritmos Quadráticos (CV $< 0{,}2\%$ em `Integer`):** Os algoritmos de classe $O(n^2)$ (*SelectionSort*, *InsertionSort* e *BubbleSort*) operando sobre `Integer` no cenário Aleatório registraram Coeficientes de Variação ($\text{CV} = (\sigma/\mu) \times 100$) inferiores a $0{,}2\%$ para $N = 100.000$ (como no *SelectionSort*, com $\mu \approx 23.870{,}97$ ms e $\sigma \approx \pm 23{,}84$ ms na Tabela 2). Os baixos coeficientes de variação indicam que, nas condições avaliadas, os tempos medidos para esses algoritmos apresentaram pouca dispersão relativa. O elevado tempo de execução também faz com que pequenas oscilações absolutas representem uma parcela reduzida da média. Esse resultado caracteriza a consistência das medições realizadas, mas não deve ser interpretado como ausência de variabilidade do ambiente de execução.
2. **Variabilidade dos Algoritmos Não Comparativos:** Em contrapartida, os algoritmos não comparativos (*BucketSort*, *RadixSort* e *CountingSort*) apresentaram variações percentuais mais expressivas. No cenário Aleatório (Tabela 2), o CV em $N = 100.000$ situou-se em $1{,}31\%$ no *RadixSort* e $2{,}46\%$ no *CountingSort*, mas

atingiu $34{,}17\%$ no *BucketSort* ($\mu = 18{,}46$ ms, $\sigma = 6{,}31$ ms). Para $N = 1.000.000$ (Aleatório), o CV foi de $1{,}48\%$ no *CountingSort*, $13{,}72\%$ no *BucketSort* e $17{,}60\%$ no *RadixSort* (observando-se variações ainda maiores no *BucketSort* nos cenários Ordenado e Invertido, onde as médias são reduzidas). A maior variabilidade do *BucketSort* pode estar relacionada à alocação dinâmica de estruturas auxiliares (`ArrayList<Integer>`) utilizadas na distribuição e ordenação interna dos elementos. Entretanto, como o gerenciamento de memória da JVM não foi monitorado diretamente, não é possível atribuir a dispersão observada a um mecanismo específico.

3. **Assimetria entre Média e Mediana:** Para algoritmos de baixa latência (*QuickSort* e *MergeSort*), nota-se em determinados cenários uma leve assimetria positiva ($\mu > Q_2$). Nas medições para $N = 100.000$ no *QuickSort* com entradas de `String` repetidas, por exemplo, a mediana estabeleceu-se em $6{,}02$ ms e o valor mínimo em $5{,}99$ ms, enquanto o tempo máximo atingiu $10{,}93$ ms, elevando a média para $6{,}60$ ms. Esse comportamento indica a ocorrência de picos isolados de execução (compatíveis com otimizações dinâmicas em tempo de execução da máquina virtual), evidenciando que a **mediana constitui um estimador mais robusto que a média** frente a valores extremos (*outliers*).
4. **Estabilidade das Medições:** A reduzida dispersão observada entre a mediana e a média na maioria das configurações indica boa consistência interna das medições coletadas, sugerindo que a bateria de pré-aquecimento (*warm-up*) e a execução em ambiente dedicado contribuíram para reduzir interferências externas nos dados compilados na Tabela 2.

4.5. Análise de Perfil Individual de Desempenho

Para compreender a estabilidade interna, é produtivo representar algumas classes de algoritmos e observar sua curva em diferentes configurações de entrada simultaneamente.

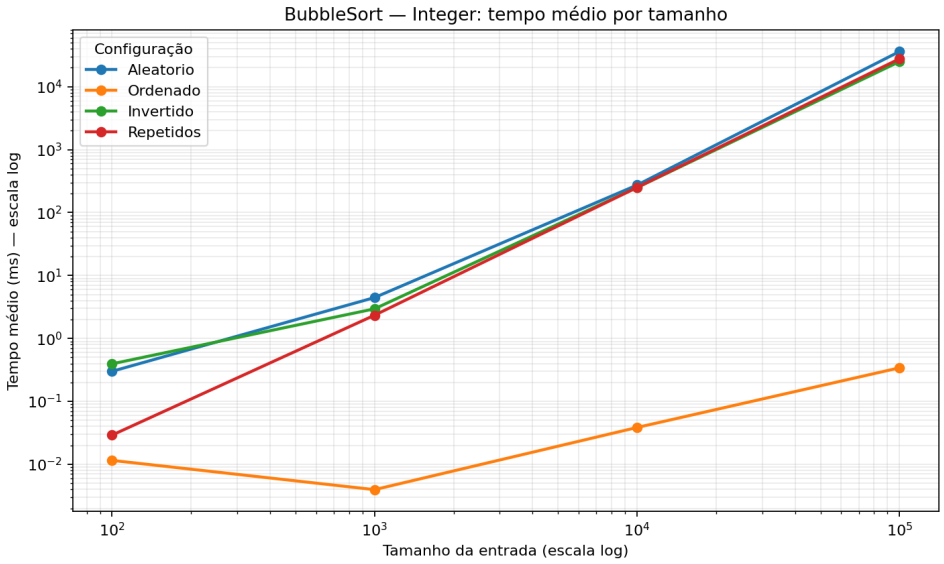


Figura 10. Variação de tempo do *Bubble Sort* perante todas as configurações de entrada.

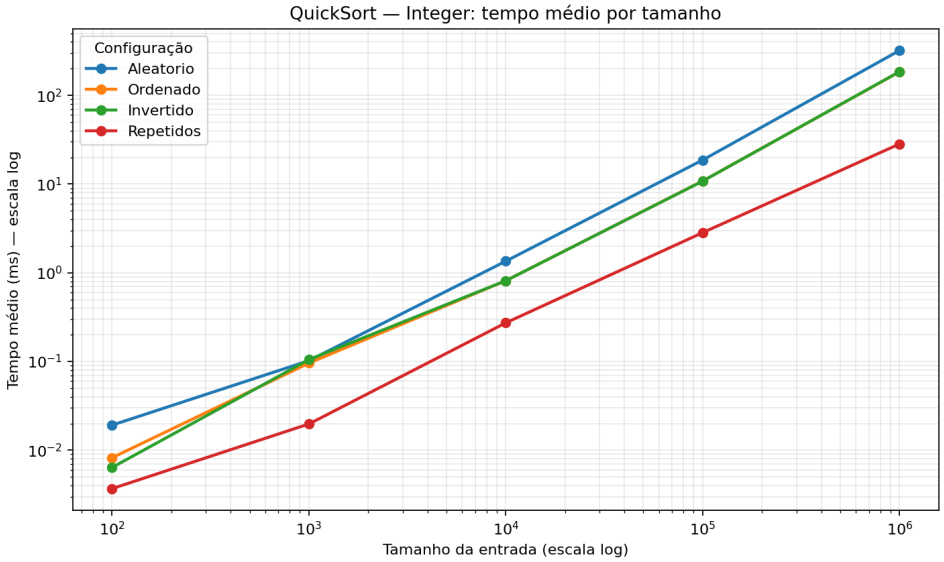


Figura 11. Variação de tempo do *Quick Sort* perante todas as configurações de entrada.

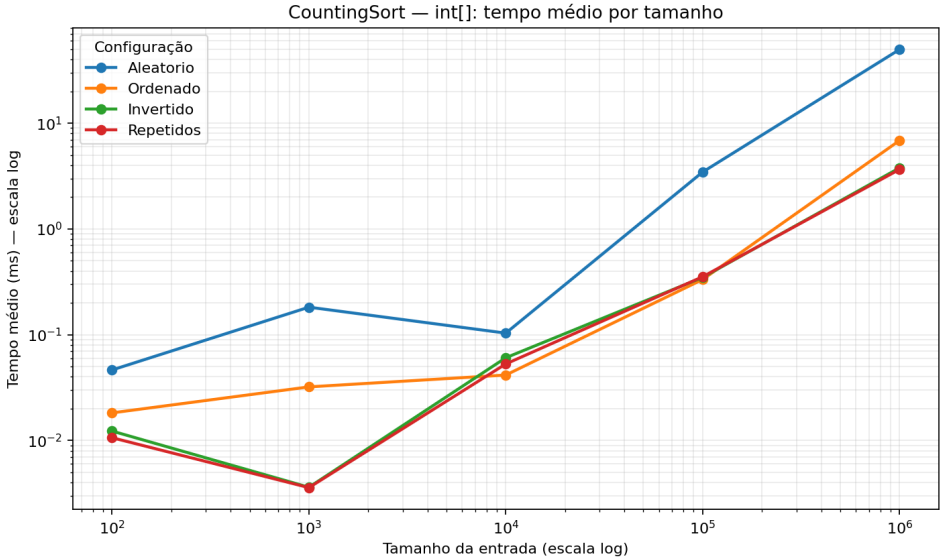


Figura 12. Variação de tempo do *Counting Sort* perante todas as configurações de entrada.

As Figuras 10 e 11 traçam o perfil do *BubbleSort* e do *QuickSort* sob os quatro cenários testados. No perfil individual do *BubbleSort*, observa-se a separação das linhas: enquanto a curva representativa da entrada "Ordenado" mantém-se na base do gráfico (melhor caso $O(n)$), as linhas de "Aleatório" e "Invertido" apresentam crescimento acentuado, compatível com o comportamento quadrático $O(n^2)$. Para o *QuickSort*, nota-se um agrupamento coeso entre os cenários ordenado, invertido e aleatório, demonstrando que, nas configurações avaliadas, a escolha do pivô central apresentou bom comportamento prático

e não levou à degradação observada no pior caso teórico $O(n^2)$. O cenário "Repetidos" apresenta queda expressiva de tempo graças ao particionamento em três vias.

Por fim, a Figura 12 ilustra o comportamento do *CountingSort* operando sobre tipos primitivos. Diferente dos algoritmos baseados em comparação, as suas curvas caminham aglomeradas na base do gráfico, evidenciando a sua eficiência linear $O(n + k)$ independentemente do estado inicial dos dados.

É interessante notar, contudo, uma anomalia (uma "queda" no tempo de execução) nas transições dos tamanhos iniciais ($N \leq 10^4$). Apesar da aplicação metodológica de 3 rodadas de *warm-up*, as execuções nestas ordens de grandeza ocorrem em frações de milissegundo. Uma possível explicação para essa variação está no comportamento adaptativo da JVM, incluindo a compilação JIT e suas otimizações em tempo de execução. Entretanto, como essas etapas internas não foram monitoradas diretamente durante o experimento, essa interpretação deve ser considerada uma hipótese, e não uma causa comprovada.

Nota: Por restrições de formatação e concisão deste relatório, apresentamos apenas os perfis emblemáticos do Bubble Sort, Quick Sort e Counting Sort para representar algumas das complexidades. Os gráficos individuais completos para todos os dez algoritmos, sob todas as permutações e cenários, encontram-se disponíveis no repositório do projeto no GitHub.

4.6. Análise de Anomalias e Padrões Não-Óbvios

A análise conjunta das métricas da Tabela 2 e dos perfis individuais destaca padrões práticos relevantes observados nos testes:

- **Comportamento do *SelectionSort* nas Configurações Testadas:** Diferente de métodos adaptativos, o *SelectionSort* apresentou tempos de execução muito próximos entre si nas diferentes configurações avaliadas (19,3 s a 24,0 s para $N = 100.000$ em `Integer`), o que corrobora o comportamento esperado da execução de $\frac{n(n-1)}{2}$ comparações independentemente da ordem inicial dos dados.
- **Desempenho do *InsertionSort* em Baixa Entropia:** Nos cenários pré-ordenados para $N = 100.000$ em `Integer`, o *InsertionSort* registrou tempos expressivamente baixos ($\approx$ 0,43 ms), superando algoritmos de complexidade assintótica superior como o *MergeSort* ($\approx$ 8,78 ms) e o *QuickSort* ($\approx$ 10,82 ms) nessas condições específicas de entrada ordenada.
- **Eficiência do *ShellSort* em Entradas Invertidas e Ordenadas:** Nos testes com `Integer` em $N = 100.000$, o *ShellSort* apresentou latências inferiores às de algoritmos clássicos como *QuickSort* e *MergeSort* nos cenários ordenado ($\approx 4{,}40$ ms) e invertido ($\approx 7{,}24$ ms), indicando que os saltos decrescentes mitigaram de forma eficiente a movimentação de elementos distantes.
- **Variação de Desempenho no *QuickSort* com Chaves Repetidas:** A introdução de alta densidade de chaves duplicadas no intervalo $[0, 10)$ reduziu o tempo médio do *QuickSort* em `Integer` para $N = 100.000$ de 18,67 ms (caso aleatório) para 2,83 ms (caso repetido), resultado associado ao particionamento de 3 vias implementado, que agrupa chaves iguais de forma eficiente.

- **Impacto da Complexidade de Comparação em Objetos `String`:** Conforme a Tabela 2 ($N = 100.000$), a substituição de tipos `Integer` por objetos `String` elevou a latência do *QuickSort* em $\approx 93{,}6\%$ (de $18{,}67$ ms para $36{,}15$ ms), enquanto o *BubbleSort* apresentou um incremento de $\approx 145{,}4\%$ (de $36.289{,}66$ ms para $89.069{,}35$ ms). Esse comportamento sugere que o custo adicional da chamada ao método `compareTo()` em cadeias de caracteres atua de forma proporcionalmente mais severa sobre algoritmos com maior volume total de comparações.
- **Sensibilidade do *BucketSort* à Distribuição dos Valores:** Para $N = 100.000$ (Figura 8), o *BucketSort* apresentou tempos distintos entre os cenários avaliados: $18{,}46$ ms em entradas aleatórias, $2{,}80$ ms em entradas invertidas, $4{,}82$ ms em entradas ordenadas e $8{,}90$ ms em entradas repetidas. Os resultados estão relacionados à distribuição dos elementos entre os baldes e à necessidade de ordenação interna. Nas entradas ordenadas e invertidas, a distribuição uniforme dos valores permitiu que cada balde recebesse um único elemento, dispensando a ordenação interna. Já nas entradas repetidas, a concentração de valores em apenas 10 baldes exigiu o uso do *MergeSort* auxiliar sobre listas de ≈ 10.000 elementos. Esses resultados ilustram como a distribuição dos dados influencia diretamente o desempenho prático dessa implementação.

5. Conclusões

Este trabalho avaliou experimentalmente dez algoritmos de ordenação implementados em Java, considerando diferentes tamanhos de entrada, tipos de dados e configurações dos elementos. O objetivo foi relacionar os tempos de execução observados às características das implementações e às complexidades temporais discutidas na análise teórica.

Os resultados permitem comparar o comportamento dos algoritmos nos cenários avaliados e observar que a escolha do método de ordenação influencia o desempenho, especialmente à medida que o tamanho da entrada aumenta. Algoritmos de complexidade quadrática tendem a apresentar limitações em entradas grandes, enquanto métodos com complexidade ($O(n \log n)$) podem oferecer melhor escalabilidade. Essa tendência, contudo, deve ser interpretada considerando os casos específicos, as características das entradas e os detalhes de cada implementação.

Os algoritmos não comparativos apresentaram vantagens de desempenho em configurações específicas, especialmente quando as características das chaves favoreceram suas estratégias de processamento. O Counting Sort demonstrou eficiência em domínios numéricos de amplitude reduzida, enquanto o Radix Sort apresentou desempenho competitivo em entradas de maior amplitude, obtendo os menores tempos médios entre os algoritmos não comparativos no cenário Aleatório para os maiores tamanhos avaliados. O Bucket Sort, por sua vez, mostrou-se sensível à distribuição dos elementos entre os baldes e ao custo da ordenação interna, realizada por meio do Merge Sort. Dessa forma, os resultados evidenciam que a eficiência desses métodos depende tanto de suas propriedades teóricas quanto das características das entradas e das decisões de implementação.

A comparação entre diferentes tipos de dados também evidencia a importância das operações realizadas durante a ordenação. Quando os elementos são objetos, o custo de comparação e o comportamento da implementação podem influenciar o tempo total, além do número de operações previsto pela análise assintótica.

Por fim, os resultados devem ser compreendidos dentro dos limites do ambiente e do protocolo experimental utilizados. A média, a mediana e o desvio padrão ajudam a caracterizar os tempos medidos, mas não eliminam todas as possíveis fontes de variação. Como continuidade, recomenda-se repetir os experimentos após alterações nas implementações, verificar a consistência dos dados e manter disponíveis os códigos e os registros individuais de execução para favorecer a reprodutibilidade.

Conclui-se que a avaliação de algoritmos de ordenação exige a combinação entre análise teórica e evidência experimental. A complexidade assintótica fornece uma referência importante para compreender a escalabilidade, enquanto os experimentos permitem observar como as características dos dados e as decisões de implementação afetam o desempenho em condições concretas.

Disponibilidade de Repositório e Dados

O código-fonte integral das implementações, as suítes de testes de corretude, o motor automatizado de *benchmarking* e a base de dados brutos coletados neste experimento encontram-se disponíveis em repositório público para auditoria e reprodutibilidade através do link: `https://github.com/Beforg/acd-tp1`.

Declaração de Uso de Inteligência Artificial

Em conformidade com as diretrizes de integridade acadêmica e transparência na pesquisa, declara-se que ferramentas de Inteligência Artificial Generativa foram utilizadas como recurso auxiliar durante o desenvolvimento deste trabalho, restritas às seguintes atividades:

1. **Apoio e Aprofundamento Teórico:** Consulta e síntese de conceitos fundamentais de análise assintótica e estruturas de dados, tomando como referência primária a literatura clássica de algoritmos [Cormen et al., 2012, Goodrich e Tamassia, 2013].
2. **Revisão e Verificação de Código:** Inspeção estática das implementações dos dez algoritmos de ordenação em Java para auxílio na identificação de gargalos de desempenho, otimização de pilha no *QuickSort* e garantia da corretude lógica.
3. **Automação de Testes de Corretude:** Auxílio na geração da classe de verificação (`TestesCorretude`), utilizada para validar o comportamento dos algoritmos sob diferentes cenários de entrada (vetores aleatórios, previamente ordenados, invertidos e com elementos repetidos).
4. **Revisão e Formatação do Relatório Técnico:** Suporte no refinamento estilístico, coesão textual, correção gramatical e estruturação do documento no padrão da Sociedade Brasileira de Computação (SBC).

Toda a concepção do experimento, execução dos benchmarks, coleta e validação dos dados estatísticos e interpretação dos resultados foram realizadas inteiramente pelos autores.

Referências

Cormen, T. H., Leiserson, C. E., Rivest, R. L., e Stein, C. (2012). *Algoritmos: Teoria e Prática*. Elsevier, 3ª edição.

Goodrich, M. T. e Tamassia, R. (2013). *Estruturas de Dados e Algoritmos em Java*. Bookman, 5ª edição.